CompoundIQ

AI-Powered Safe 3-Drug Combination Discovery

ITAI 2277 | Artificial Intelligence Resource Capstone | Spring 2026

Oyinade Balogun | Oluchi Obinna | Sharise Griggs | Nicole Marcial

https://github.com/ComfortCS50/CompoundIQ

The Story Behind CompoundIQ

Patients with complex conditions often rely on combinations of drugs, but the more drugs that are added, the greater the risk of harmful interactions. Since the number of possible three-drug combinations is extremely large, testing every option in the lab is not realistic. CompoundIQ was designed to help narrow that search by identifying promising combinations before costly and time-intensive lab testing begins.

CompoundIQ is our proposed AI system for designing and evaluating three-drug combinations. It takes a therapeutic mechanism written in plain English, generates candidate compounds, predicts both pairwise and three-way interactions, filters out unsafe options, creates new drugs, and then ranks the safest and most promising results. In our project design, this brings together natural language processing, molecular generation, and graph-based interaction modeling into one end-to-end pipeline trained on biomedical and drug-interaction datasets such as ChEMBL, DrugBank, SIDER, and related sources.

MODEL PIPELINE

End-to-End Architecture

1 Ingestion
ChEMBL
DrugBank
SIDER / STITCH

→

2 Data Processing
Nulls,
Duplicates

→

3 Feature Extraction
Feature Selection

→

4 Model Fit
JT-VAE + BioBERT
Pairwise & 3-Way GNN

→

5 Output
Top 10 safe
drug combinations

Ingestion

ChEMBL: ~35,000 drug-like molecules
DrugBank: drug interaction records
STITCH: protein-chemical interaction data
SIDER: drug side effect profiles

Data Processing

Handle nulls (mean/median/mode)
Remove anomalies and duplicates
Normalize molecular descriptors

Feature Extraction

Identify output-influencing features
(CYP enzymes, centrality, similarity)

Why JT-VAE?

Junction Tree Variational Autoencoder for Molecular Graph Generation

How JT-VAE Works

Unlike other molecular generators that build molecules that risk invalid structure, JT-VAE uses this approach:

1 A molecule is decomposed into a junction tree of valid chemical substructures (fragments) extracted from training data

2 The tree and molecular graph are encoded into separate latent vectors (z_T and z_G)

3 To generate, the model first reconstructs the junction tree, then assembles the fragments back into a complete, valid molecule

This structure-by-structure approach guarantees every generated molecule is chemically valid

Why It Matters: Validity Comparison

Method	Reconstruction	Validity
CVAE	44.6%	0.7%
GVAE	53.7%	7.2%
SD-VAE	76.2%	43.5%
GraphVAE	—	13.5%
Atom-by-Atom LSTM	—	89.2%
JT-VAE (Ours)	76.7%	**100.0%**

100%

molecular validity — every molecule JT-VAE generates is chemically valid by construction

USER PIPELINE

How the User Interacts

User Input

Enters therapeutic mechanism in natural language

"Anti-inflammatory targeting COX-2"

→

BioBERT Encodes

Text ⟶ 768-D embedding ⟶ 56-D mapped

→

JT-VAE Generates

Creates 150 novel molecular candidates

→

GNNs Score

Pairwise + 3-Way GNN filter and rank for safety

→

Top 10 Returned

Safest drug combinations displayed

Key: The user never handles raw data. They describe a therapeutic goal in plain English, and the pipeline handles molecular generation, interaction analysis, and safety ranking automatically.

Libraries & Tools

Library	Role in CompoundIQ	Component
torch-geometric	Graph neural network layers and message-passing for drug interaction graphs	Pairwise & 3-Way GNN
RDKit	SMILES parsing, molecular validation, junction tree decomposition into fragments	JT-VAE
NetworkX	Building drug interaction graphs and junction tree structures	JT-VAE, GNN
Transformers (HF)	Loading and running BioBERT (dmis-lab/biobert-base-cased-v1.1)	BioBERT
DeepChem	Molecular dataset loading (Tox21, ESOL, BBBP) and featurization utilities	Data Collection
chembl-client	Python API client for querying ChEMBL molecular database directly	Data Collection
Streamlit	Interactive web dashboard for user input and results display	Frontend
Py3Dmol	3D molecular structure rendering embedded in the Streamlit UI	Frontend

CompoundIQ

ANY QUESTIONS?

5,344,397 trainable parameters

5-stage end-to-end pipeline

Novel molecular generation via JT-VAE

Databases & Datasets

ChEMBL

~35,000 drug-like molecules queried via API (subset of 2.4M). Primary training data for JT-VAE molecular generation with 534-fragment vocabulary.

TWOSIDES

~1M drug combination adverse event records. Polypharmacy side effect data used to train the GNN safety prediction models.

SIDER

1,430 drugs with side effect profiles. Used to enrich drug safety features and validate interaction predictions.

STITCH

Protein-chemical interaction data

DrugBank

Drug-drug interaction records. Source for Nicole's engineered features: molecular similarity, graph centrality, CYP enzyme overlap, and interaction severity scores.

All datasets are publicly available, free for academic use, and contain no patient-identifiable information.

DrugBank Feature Engineering

Nicole’s DrugBank data engineering provides the GNN with rich, domain-informed features that go beyond raw molecular structure.

Molecular Similarity

Tanimoto coefficient between drug fingerprints; structurally similar drugs often share interaction profiles.

Graph Centrality

How connected a drug is within the DrugBank interaction network; high-centrality drugs have more known interactions.

CYP Enzyme Features

Cytochrome P450 enzyme overlap between drugs; shared CYP metabolism pathways increase interaction risk.

Interaction Severity

Historical severity scores from DrugBank; encodes whether known pairwise interactions are minor, moderate, or major.

References

ChEMBL Database: Zdrazil et al. (2024); Davies et al. (2015)
DrugBank Database: Knox et al. (2024); Wishart et al. (2018)
SIDER Side Effect Resource: Kuhn et al. (2016); Kuhn et al. (2010)
STITCH Protein–Chemical Interaction Database: Szklarczyk et al. (2016)
JT-VAE Molecular Generation Model: Jin, Barzilay, & Jaakkola (2018)

Full references are available in the project GitHub `CITATIONS.md` file.